

A silhouette of a gazebo with a flag on top, set against a twilight sky. The gazebo is dark, and the sky transitions from a deep blue at the top to a lighter, hazy orange near the horizon. In the background, some distant city lights and a street lamp are visible.

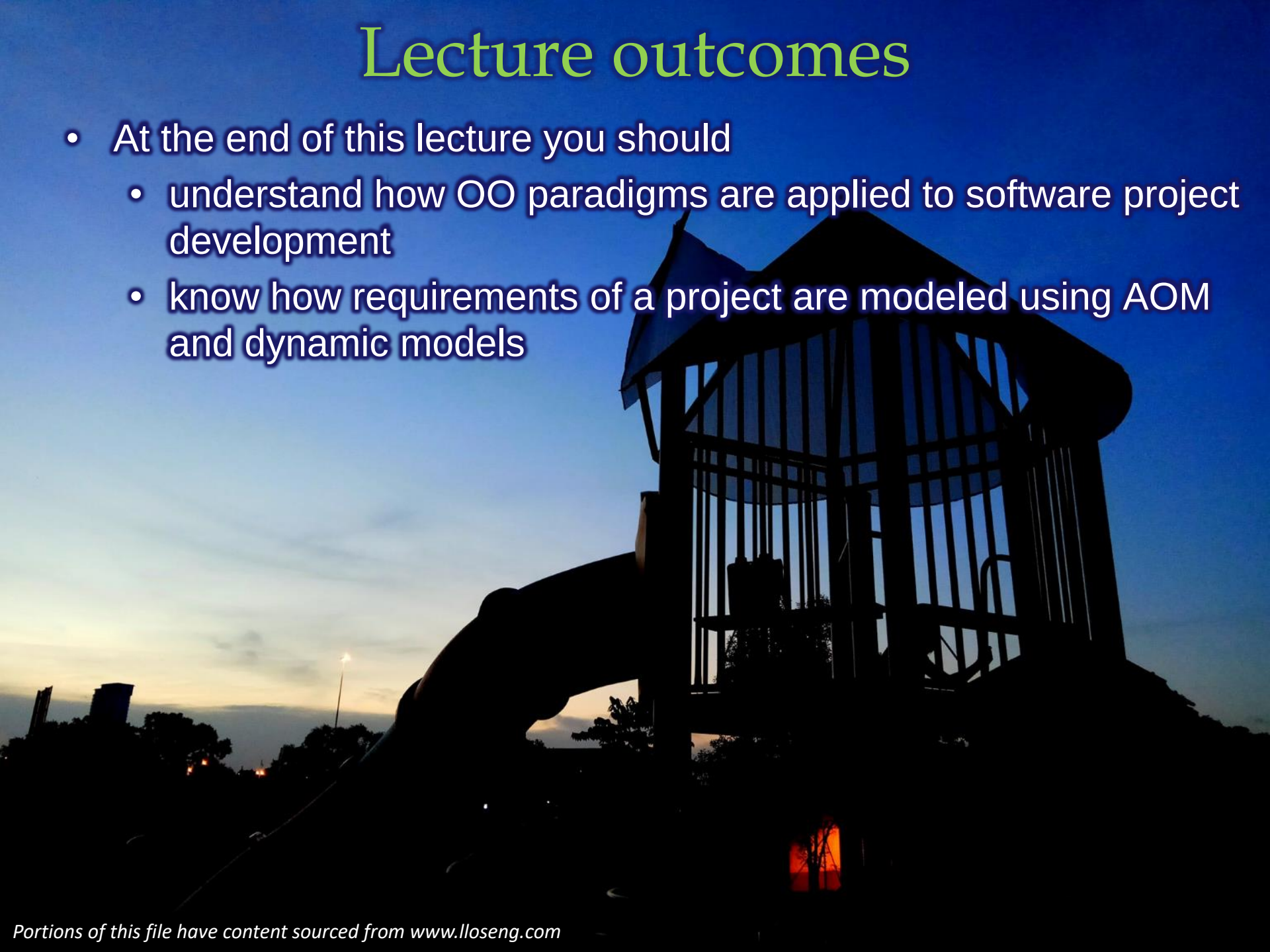
OO Analysis

TCP2201

Object-oriented Analysis and Design

Lecture outcomes

- At the end of this lecture you should
 - understand how OO paradigms are applied to software project development
 - know how requirements of a project are modeled using AOM and dynamic models



The nature of software

- Software :-
 - Is intangible – hard to quantify and difficult to understand development effort required
 - Is easy to reproduce – compared to other outputs (e.g. manufacturing) cost is purely in development
 - Is labour-intensive – hardly ever automated, mostly custom
 - Quality is difficult to maintain– multiple people working together will generate different levels of software quality
 - Is easy to modify – changes are made without understanding the problem
 - Never wears out – unlike a physical item, software will not deteriorate until its functionality is changed or is outdated
- Much software has poor design and is getting worse whilst the high demand for software continues to rise
- People should learn how to ‘engineer’ and not ‘write’ software

Types of Software

- **Custom** - for a specific customer
- **Generic**
 - Sold on open market for ready use
 - COTS (Commercial Off The Shelf)
 - Shrink-wrapped
- **Embedded** - Built into hardware and hard to change
- **Differences :-**

	Custom	Generic	Embedded
Number of <i>copies</i> in use	low	medium	high
Total <i>processing power</i> devoted to running this type of software	low	high	medium
Worldwide annual <i>development effort</i>	high	medium	low

Software engineering?

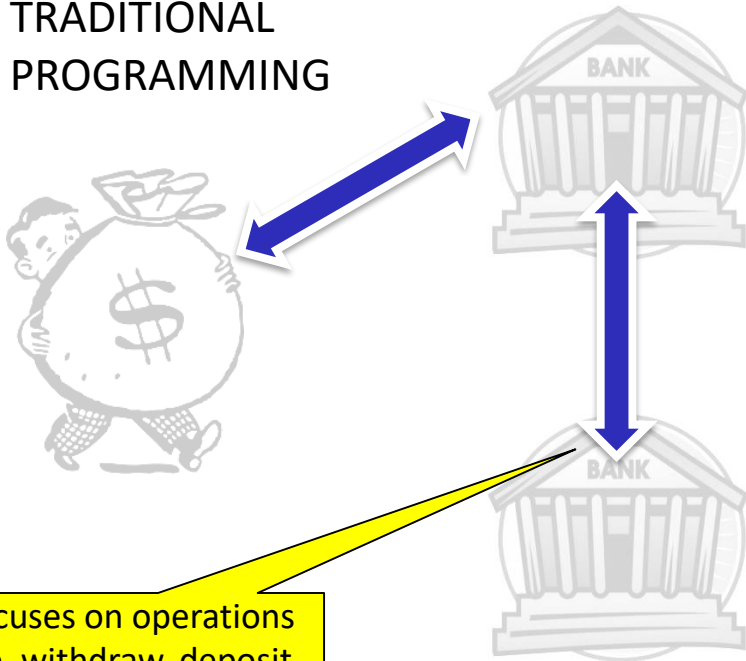
The process of solving customers' problems by the systematic development and evolution of large, high-quality software systems within cost, time and other constrain

- Solving customers' problems – the goal of software engineering
 - adding unnecessary features may not solve the problem, often times the solution is to buy and not build
- Systematic development and evolution - involves applying *well understood techniques* in a organized and *disciplined* way
- Large, high quality software systems – many systems cannot be completely understood by one person
 - Teamwork and co-ordination are required so the challenge is dividing the work and ensuring the parts work properly together
- Cost, time and other constraints – is bounded by finite resources
 - Inaccurate estimates of cost and time have caused many project failures

OO software development

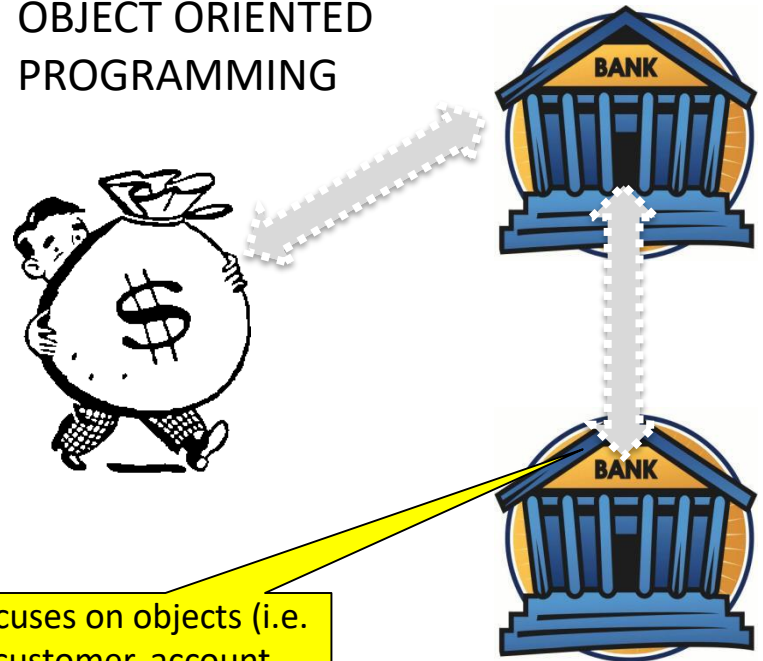
- OO represents one of many methods that can be applied in software engineering to formulate a structured plan to solve a software related problem.
- As opposed to thinking what functions the system needs, it looks at what objects are required instead and what responsibilities each object possesses.

TRADITIONAL PROGRAMMING



focuses on operations
(i.e. withdraw, deposit,
transfer)

OBJECT ORIENTED PROGRAMMING



focuses on objects (i.e.
customer, account,
branch)

Why OO?

- While developing using OO paradigm, developers/engineers should take advantage of the reusability aspects of an OO language
- Avoid re-developing software already developed
- Types of reuse:
 - Reuse of expertise (developer/engineer/designer)
 - Reuse of standard designs and algorithms
 - Reuse of libraries of classes or procedures
 - Reuse of powerful commands built into languages and/or operating systems
 - Reuse of frameworks and design patterns
 - Reuse of complete applications
- *Reuse* AND *design for reusability* should be part of the culture of software development organizations

Types of software projects

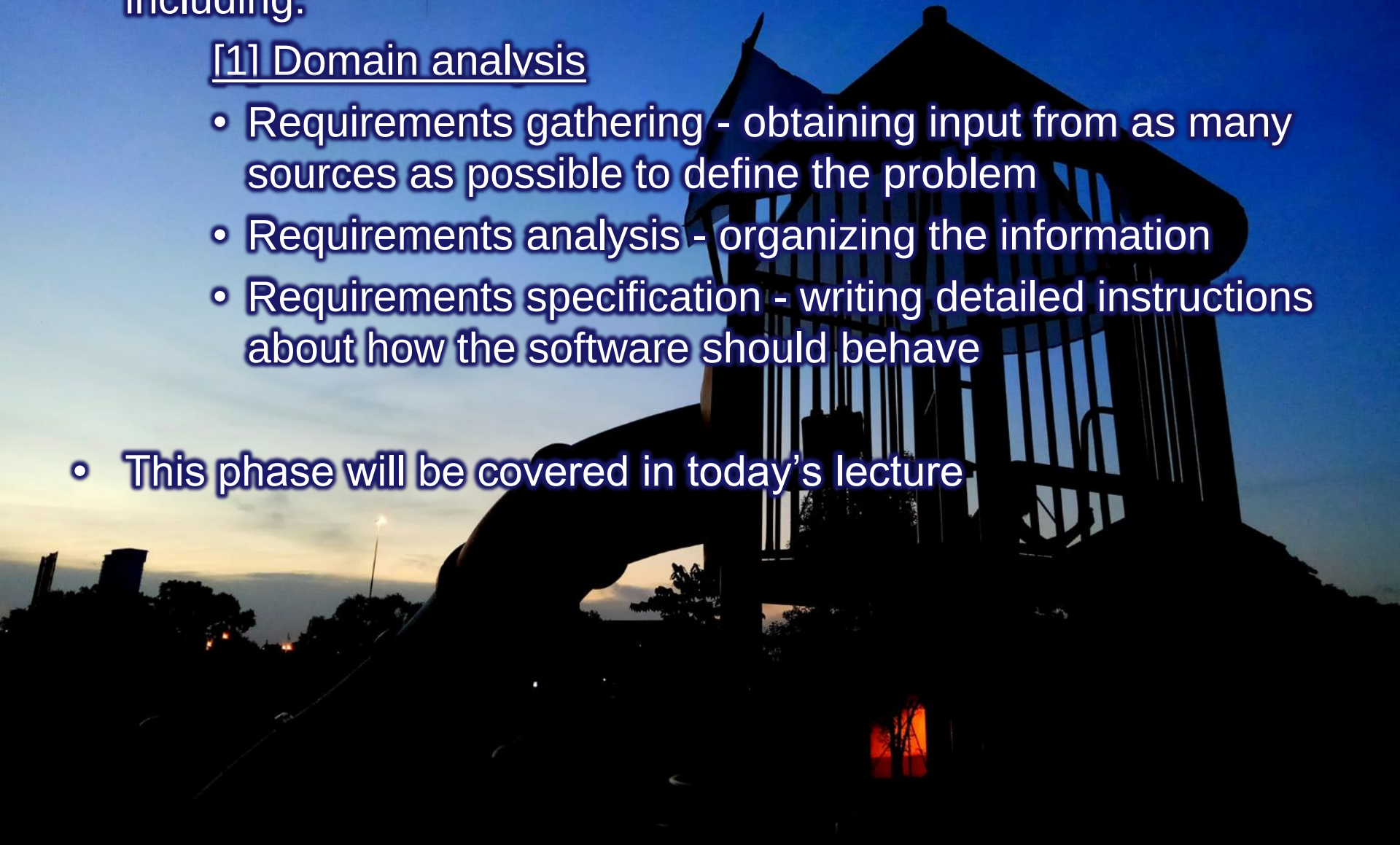
- Many projects are evolutionary or maintenance projects, involving work on legacy systems
 - **Corrective projects**: fixing defects
 - **Adaptive projects**: changing the system in response to changes in operating system, database, business rules and regulations etc.
 - **Enhancement projects**: adding new features for users
 - **Re-engineering or perfective projects**: changing the system internally so it is more maintainable – functionally it works the same
- **'Green field' projects**
 - Totally new development
 - The minority of projects

Activities in software projects

- Typical requirements and specification navigates 6 phases including:

[1] Domain analysis

- Requirements gathering - obtaining input from as many sources as possible to define the problem
 - Requirements analysis - organizing the information
 - Requirements specification - writing detailed instructions about how the software should behave
- This phase will be covered in today's lecture



[2] Design

- Deciding how the requirements should be implemented, using the available technology. Includes:
 - *Systems engineering*: Deciding what should be in hardware and what in software
 - *Software architecture*: Dividing the system into subsystems and deciding how the subsystems will interact
 - *Detailed design* of the internals of a subsystem
 - *User interface and database design*

[3] Modeling

- Creating representations of the domain or the software using UML diagrams

[4] Programming / prototyping / versioning

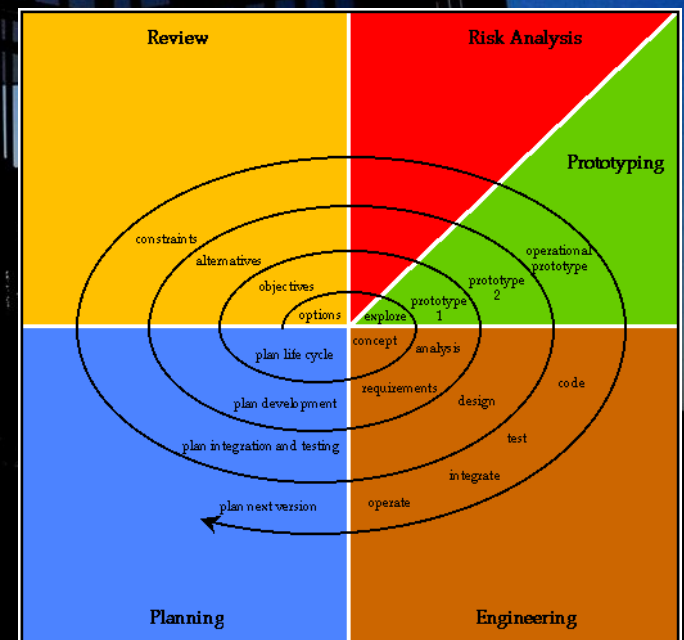
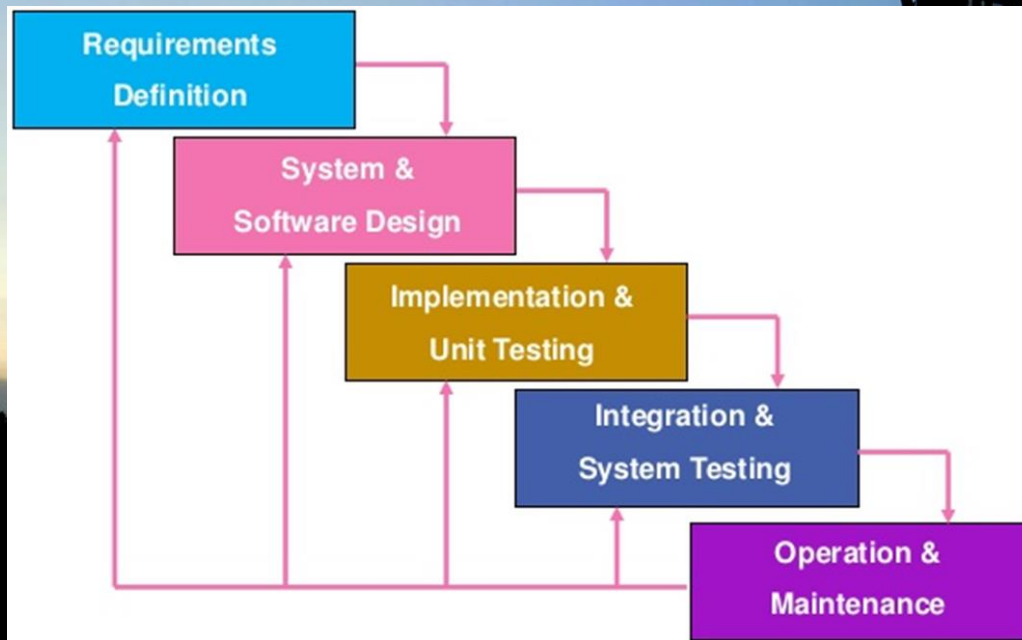
[5] Quality assurance

- Reviews and inspections
- Testing

[6] Deployment/Maintenance

How does this map to design models?

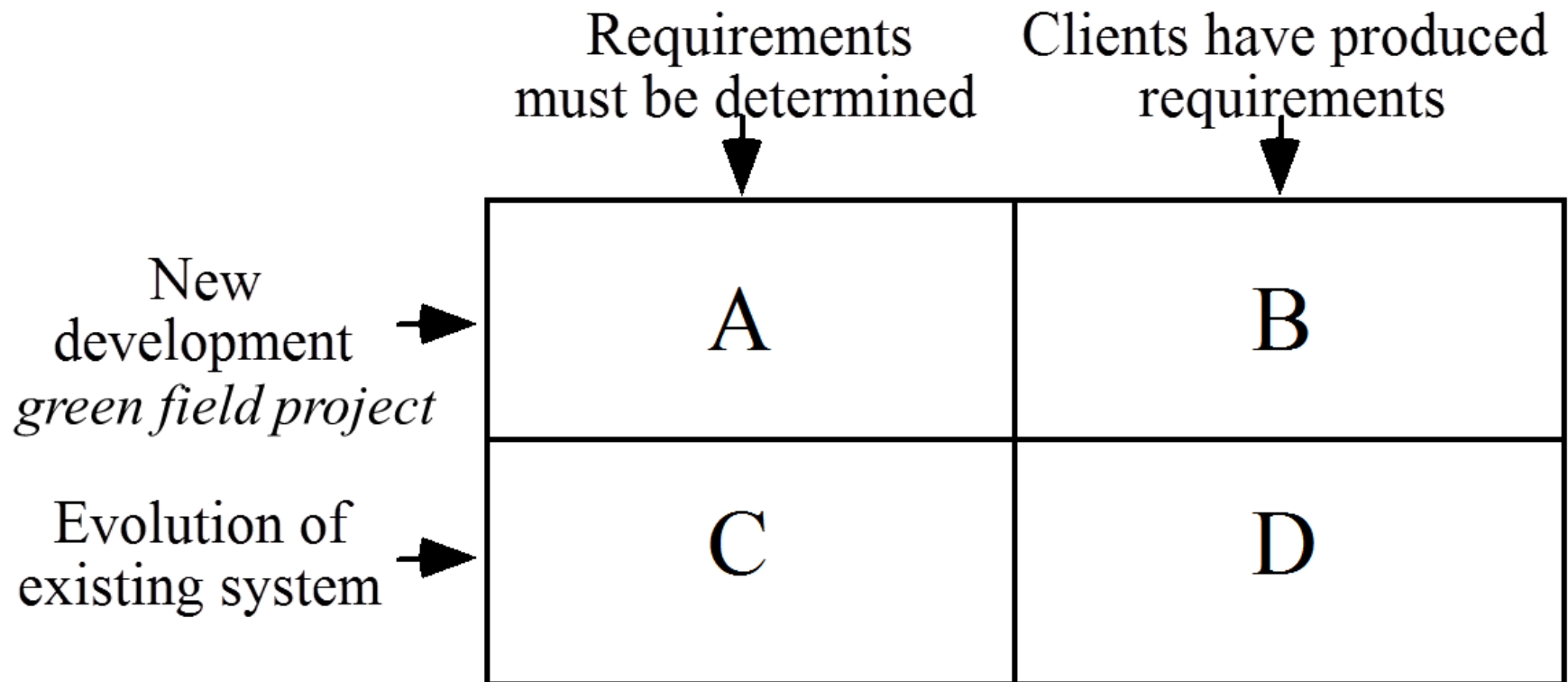
- A different course may cover content such as waterfall, V, spiral, Boehm etc.
- These models (or methodologies) are generic and work with both traditional and OO development styles
- OOAD is complementary to software design and is not meant to replace it



Domain analysis

- The software engineer learns about the domain to better understand the problem
 - The domain is the **general field of business or technology in which the customer/clients will use the software**
 - A domain expert is a **person who has a deep knowledge of the domain** – usually a well-versed client or possibly another software engineer!
- Benefits of performing domain analysis:
 - Faster development
 - Better system (less errors)
 - Anticipation of extensions (preparation for future)

The starting point of software projects

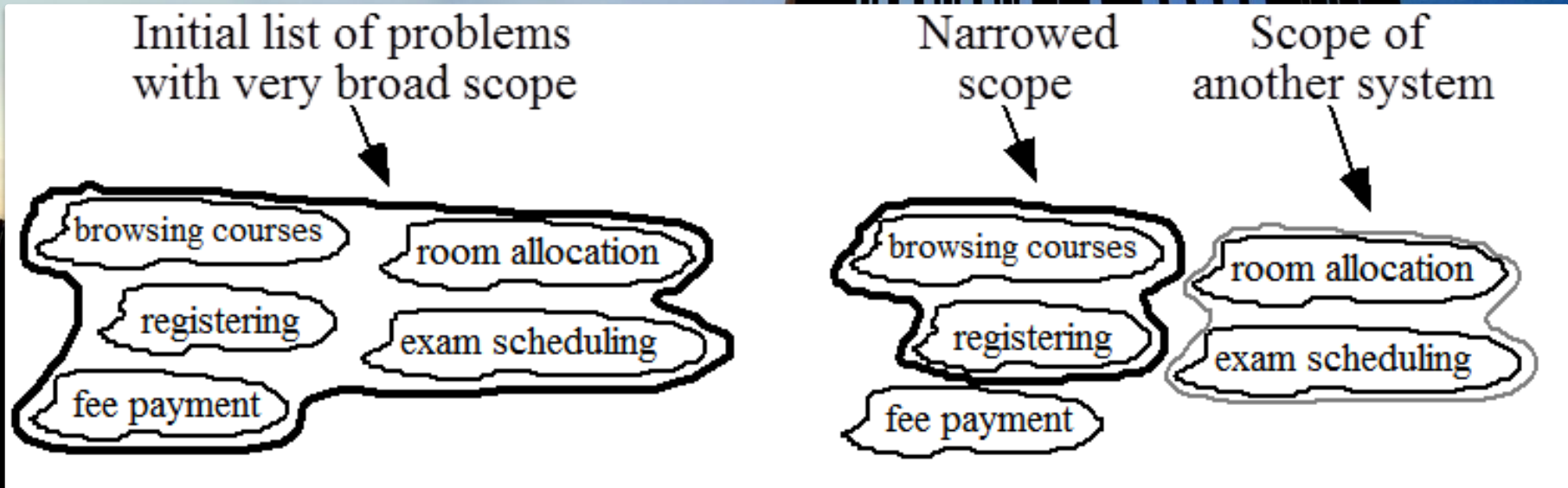


Problem definition

- A problem can be expressed as:
 - A difficulty the users or customers are facing
 - Or as an opportunity that will result in some benefit such as improved productivity or sales
- The solution to the problem normally will entail developing software – that's what we're here for!
- A good problem statement is short and succinct
- It should be straight to the point and un-ambiguous
- The definition should cover ONE (1) problem (or at least can treat multiple smaller ones and one large one) – else there may be multiple problem definitions, thus there may be multiple projects
- The problem definition will lead into discovering the project requirements

Problem scope

- Narrow the *scope* by defining a more precise problem
 - List all the things you might imagine the system doing
 - Exclude some of these things if too broad
 - Combine those that have reasons to work together (or should go together)
 - Determine high-level goals if too narrow (summary goals)
- Example – university registration system



Defining requirements

- What is a requirement?
- It is a statement describing either
 - [1] an aspect of what the proposed system must do or [2] a constraint on the system's development.
 - In either case it must contribute in some way towards adequately solving the customer's problem;
 - the set of requirements as a whole represents a negotiated agreement among the stakeholders.
- A collection of requirements is called a *requirements document*.
- All documents should be:
 - sufficiently complete
 - well organized
 - clear and legible
 - agreed to by all the stakeholders

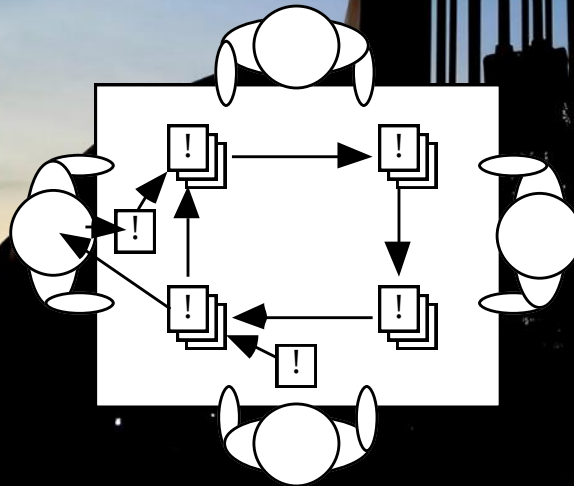
Types of requirements

- **Functional requirements**
 - Describe *what* the system should do. Examples
 - What input/outputs the system should accept/produce
 - What data the system should store that other systems might use
 - What computations the system should perform
 - The timing and synchronization of the above
- **Quality requirements**
 - *Constraints* on the design to meet specified levels of quality. Examples incl constraints on response time, throughput, resource usage, availability, recoverability etc.
- **Platform requirements**
 - *Constraints* on the environment and technology of the system
- **Process requirements**
 - *Constraints* on the project plan and development methods

Gathering requirements

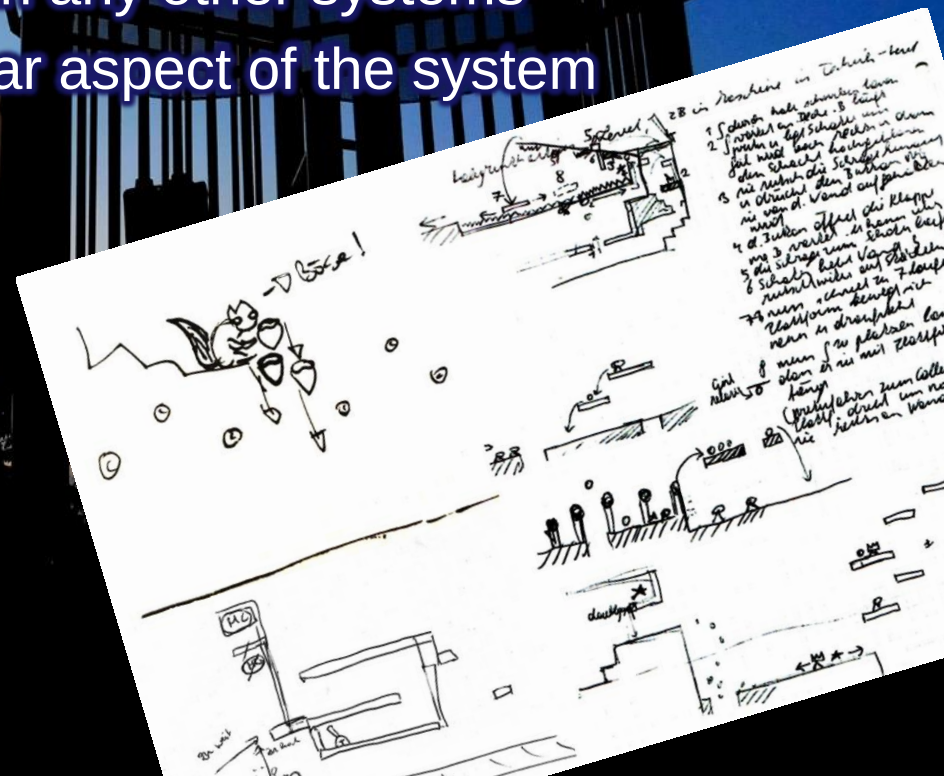
- Various methods can be used to gather requirements
- Observation
 - Read documents and discuss requirements with users
 - Shadowing important potential users as they do their work - ask the user to explain everything he or she is doing
 - Session videotaping
- Interviewing
 - Conduct a series of interviews to
 - Ask about specific details
 - Ask about the stakeholder's vision for the future
 - Ask if they have alternative ideas
 - Ask for other sources of information
 - Ask them to draw diagrams

- Brainstorming
 - Appoint an experienced moderator
 - Arrange the attendees around a table
 - Decide on a 'trigger question'
 - Ask each participant to write an answer and pass the paper to its neighbour to evaluate
 - *Joint Application Development (JAD)* is a technique based on intensive brainstorming sessions



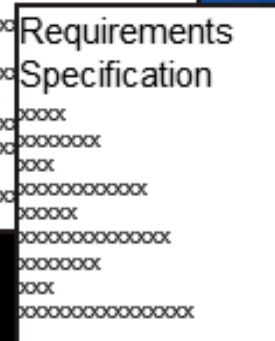
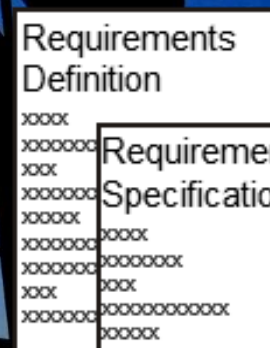
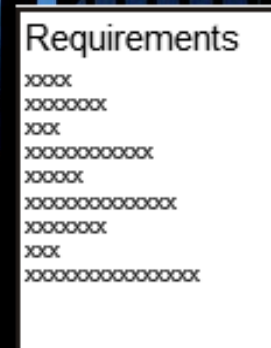
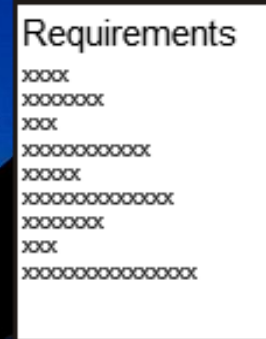
- Prototyping

- The simplest kind: paper prototype - a set of pictures of the system that are shown to users in sequence to explain what would happen
- The most common: a mock-up of the system's UI written in a rapid prototyping language
- Does not normally perform any computations, access any databases or interact with any other systems
- May prototype a particular aspect of the system



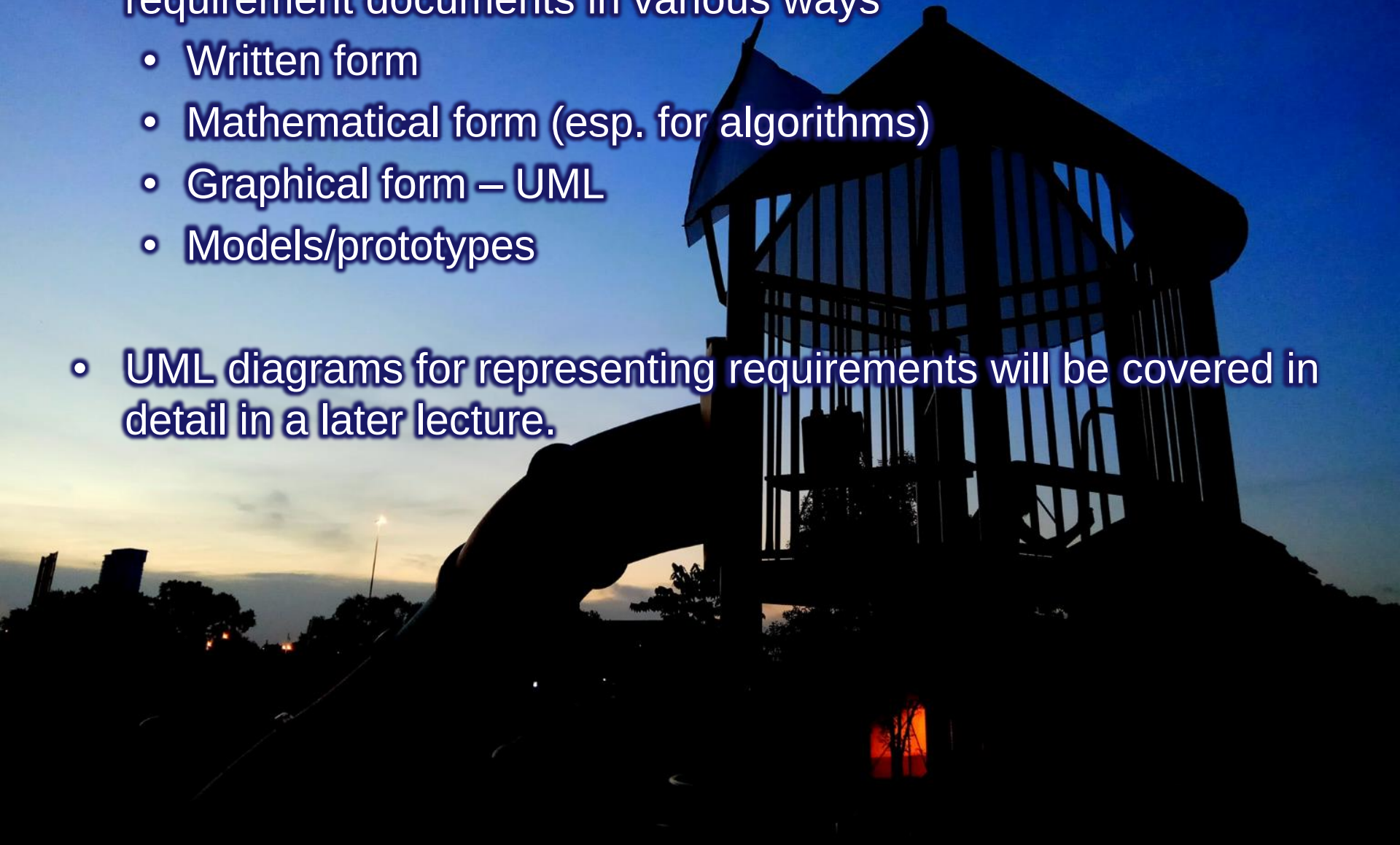
Types of requirements document

- **Requirements definition**
An informal outline of the requirements using a few paragraphs or simple diagrams
- **Requirements specification**
A long list of specifications that contain thousands of pages of intricate detail
- Requirements documents for large systems are normally arranged in a hierarchy



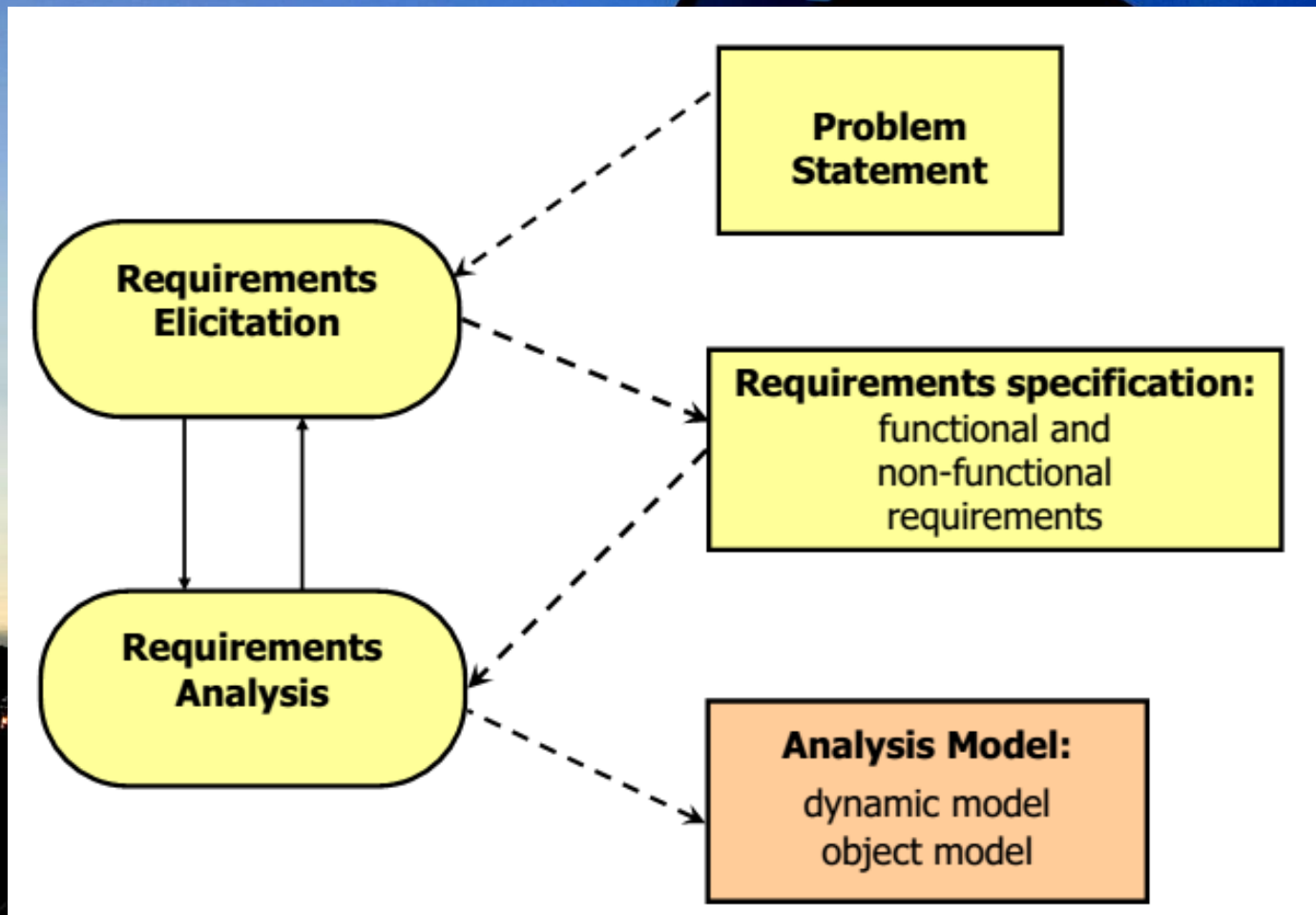
Depicting requirements

- Requirements determined in a project can be represented in the requirement documents in various ways
 - Written form
 - Mathematical form (esp. for algorithms)
 - Graphical form – UML
 - Models/prototypes
- UML diagrams for representing requirements will be covered in detail in a later lecture.

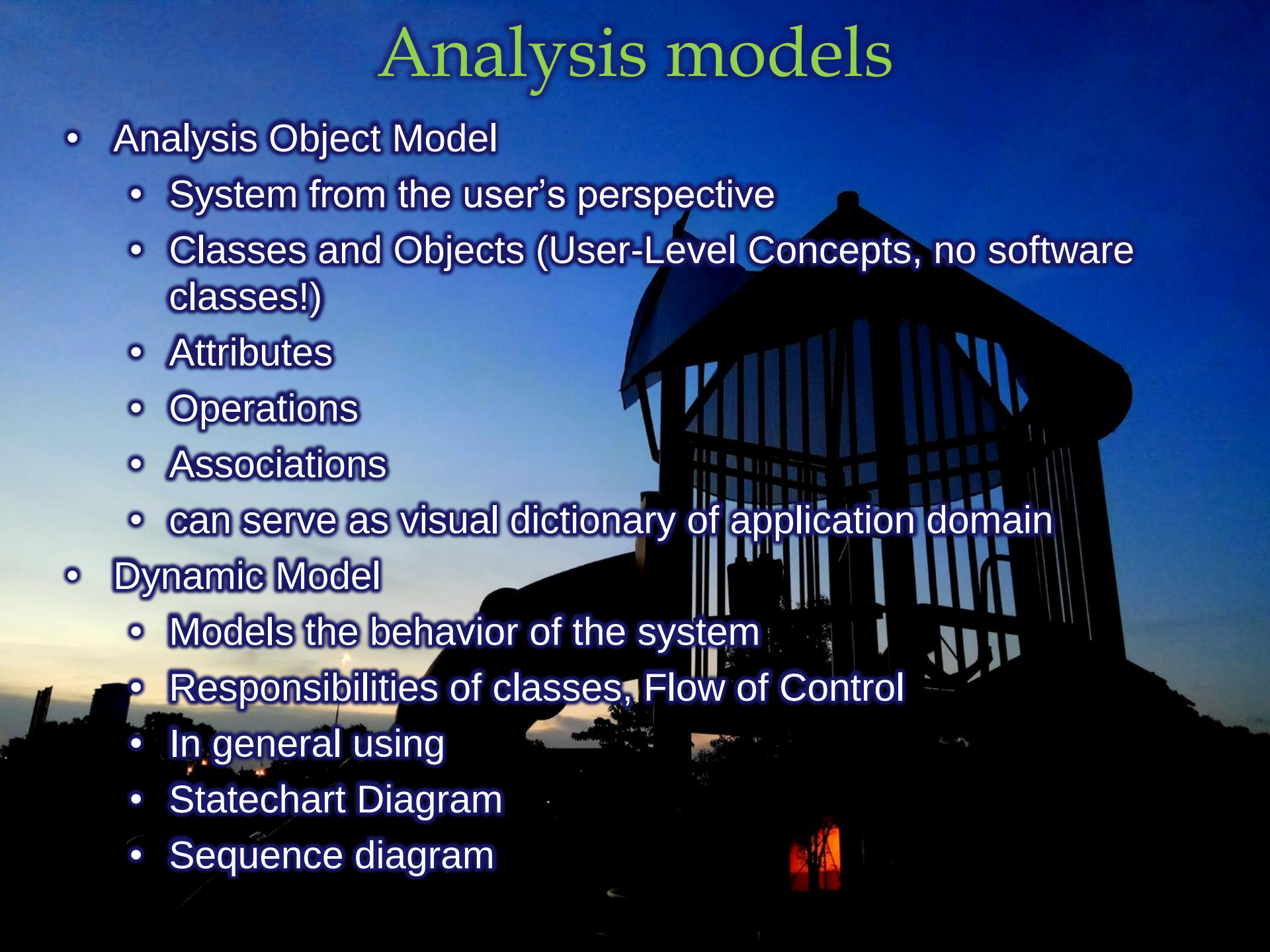


Requirements to Analysis models

- Requirements analysis takes the results of domain analysis (requirement documents) to create models to better represent the domain in a programmatic manner



Analysis models

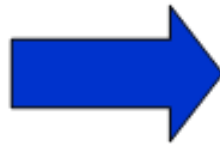
A silhouette of a gazebo with a multi-tiered roof and vertical slats, set against a twilight sky. The gazebo is positioned on the right side of the frame, with its structure clearly visible against the fading light. The sky transitions from a deep blue at the top to a lighter, hazy glow near the horizon. In the lower right, a small, warm orange light source, possibly a lamp or a fire, adds a touch of color to the dark foreground.

- Analysis Object Model
 - System from the user's perspective
 - Classes and Objects (User-Level Concepts, no software classes!)
 - Attributes
 - Operations
 - Associations
 - can serve as visual dictionary of application domain
- Dynamic Model
 - Models the behavior of the system
 - Responsibilities of classes, Flow of Control
 - In general using
 - Statechart Diagram
 - Sequence diagram

Creating the Analysis object model

Requirements Elicitation Specification

- Scenarios
- Use Cases
-
- Functional Requirements
- Conditions
- Exceptions



Analysis Object Model

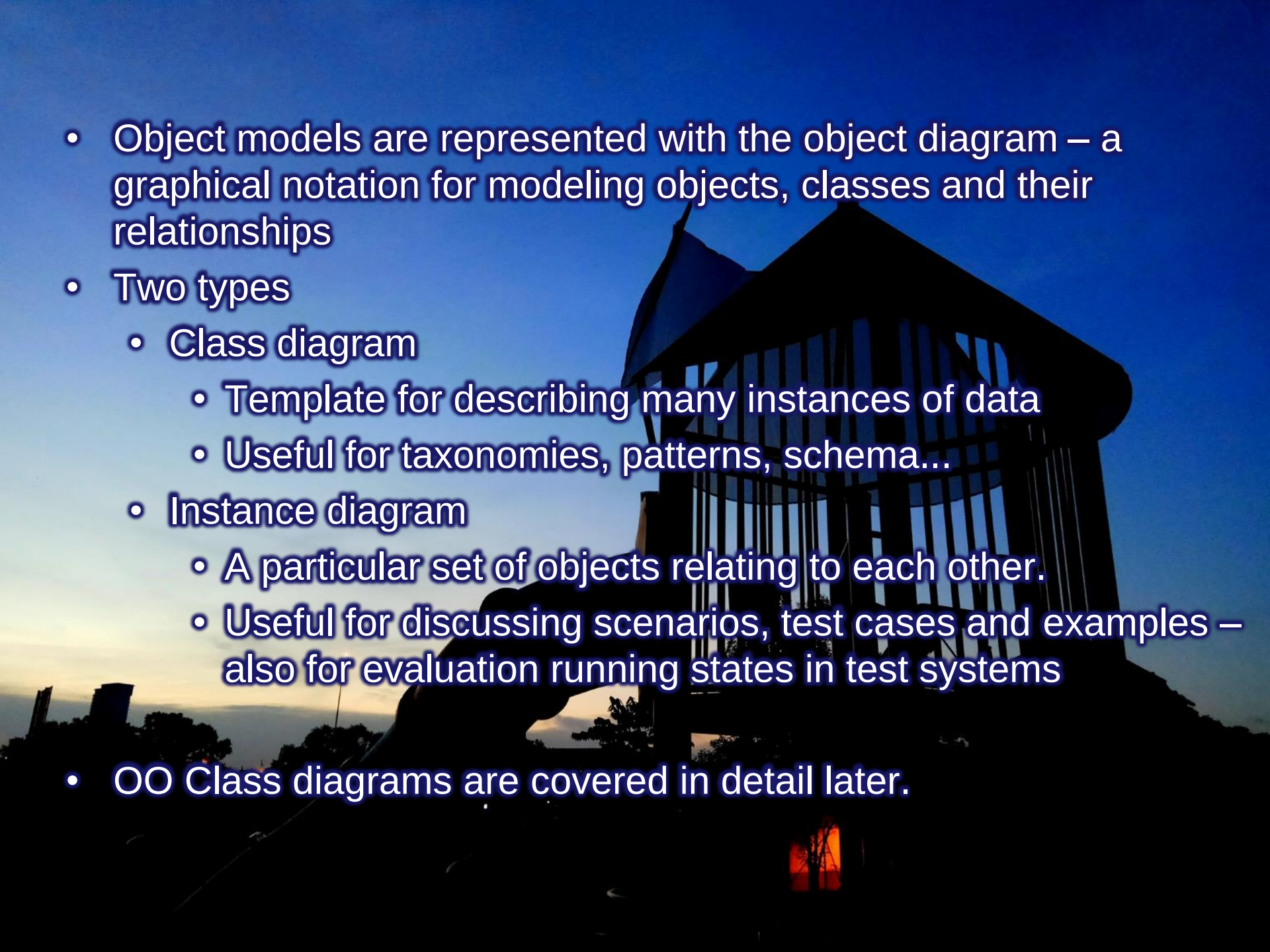
- ♦ Important Objects and Classes
- ♦ Attributes
 - ♦ Application specific
 - ♦ Attributes in one system can be classes in another
 - ♦ Turning Attributes into Classes
- ♦ Operations
 - ♦ Detection of operations
 - ♦ Generic operations: Get/Set, design patterns
 - ♦ Domain operations: Dynamic model
- ♦ Associations (Relations)
 - ♦ Generic associations
 - ♦ Canonical associations
 - ♦ Part of- Hierarchy (Aggregation)
 - ♦ Kind of-Hierarchy (Generalization)

Steps to analysis

- Formulate a few scenarios with help from the end user and/or application domain expert
- Extract the use cases from the scenarios, with the help of application domain expert
- Analyze the flow of events
- Generate the class diagrams, which includes the following steps:
 - Class identification (textual analysis, domain experts) across all Use Cases (sharing of classes and objects)
 - Identification of attributes and operations (sometimes before the classes are found!)
 - Identification of associations between classes
 - Identification of multiplicities
 - Identification of roles
 - Identification of constraints

Class Identification

- Identify and describe all important entities in the system
- Steps during object modeling
 - Class identification – identifying objects important to the sys
 - Find the attributes and methods
 - Find the associations between classes
- Iterate and correct the model (after reanalysis/checking)
- Objects are not just found by taking a picture of a scene or domain - the application domain has to be analyzed
- Depending on the purpose of the system different objects might be found
- How can we identify the purpose of a system? Take the Scenarios and use cases as foundation!
- Another important issue – define system boundary → what objects are inside and what objects are outside the system?

- 
- Object models are represented with the object diagram – a graphical notation for modeling objects, classes and their relationships
 - Two types
 - Class diagram
 - Template for describing many instances of data
 - Useful for taxonomies, patterns, schema...
 - Instance diagram
 - A particular set of objects relating to each other.
 - Useful for discussing scenarios, test cases and examples – also for evaluation running states in test systems
 - OO Class diagrams are covered in detail later.

Identifying participating objects

- Pick a use case and look at its flow of events
- Find terms that developers or users need to clarify in order to understand the flow of events
- Look for recurring nouns (e.g., *Incident*, *Account*),
- Identify real world entities that the system needs to keep track of (e.g., *FieldOfficer*, *Dispatcher*, *Resource*),
- Identify real world procedures that the system needs to keep track of (e.g., *EmergencyOperationsPlan*),
- Identify data sources or sinks (e.g., *Printer*)
- Be prepared that some objects are still missing and need to be found:
- Model the flow of events with a sequence diagram (later..)
- Always use the user's terms (i.e. based on domain)

Example

- The customer enters the store to buy a toy
- It has to be a toy that his daughter likes and it must cost less than 50 Ringgit.
- An assistant helps him and mentions that the suitability of a toy depends on the age of the child.
- The customer's daughter is 10 years old
- The assistant recommends one type of toy, namely a dress-up doll. For this game, the players must be older than 7.
- The assistant also suggests a game collection which is also a toy that consists of many separate games like chess and card-games.
- For this collection, the players must be older than 5

Example

- The **customer** enters the **store** to buy a **toy**
- It has to be a toy that his **daughter** likes and it must cost less than 50 Ringgit.
- An **assistant** helps him and mentions that the suitability of a toy depends on the age of the child.
- The customer's daughter is 10 years old
- The assistant **recommends** one **type of toy**, namely a **Barbie doll**. For this game, the players must be older than 7.
- The assistant also **suggests** a **game collection** which is also a toy that **consists of** many separate games like chess and card-games.
- For this collection, the players must be older than 5

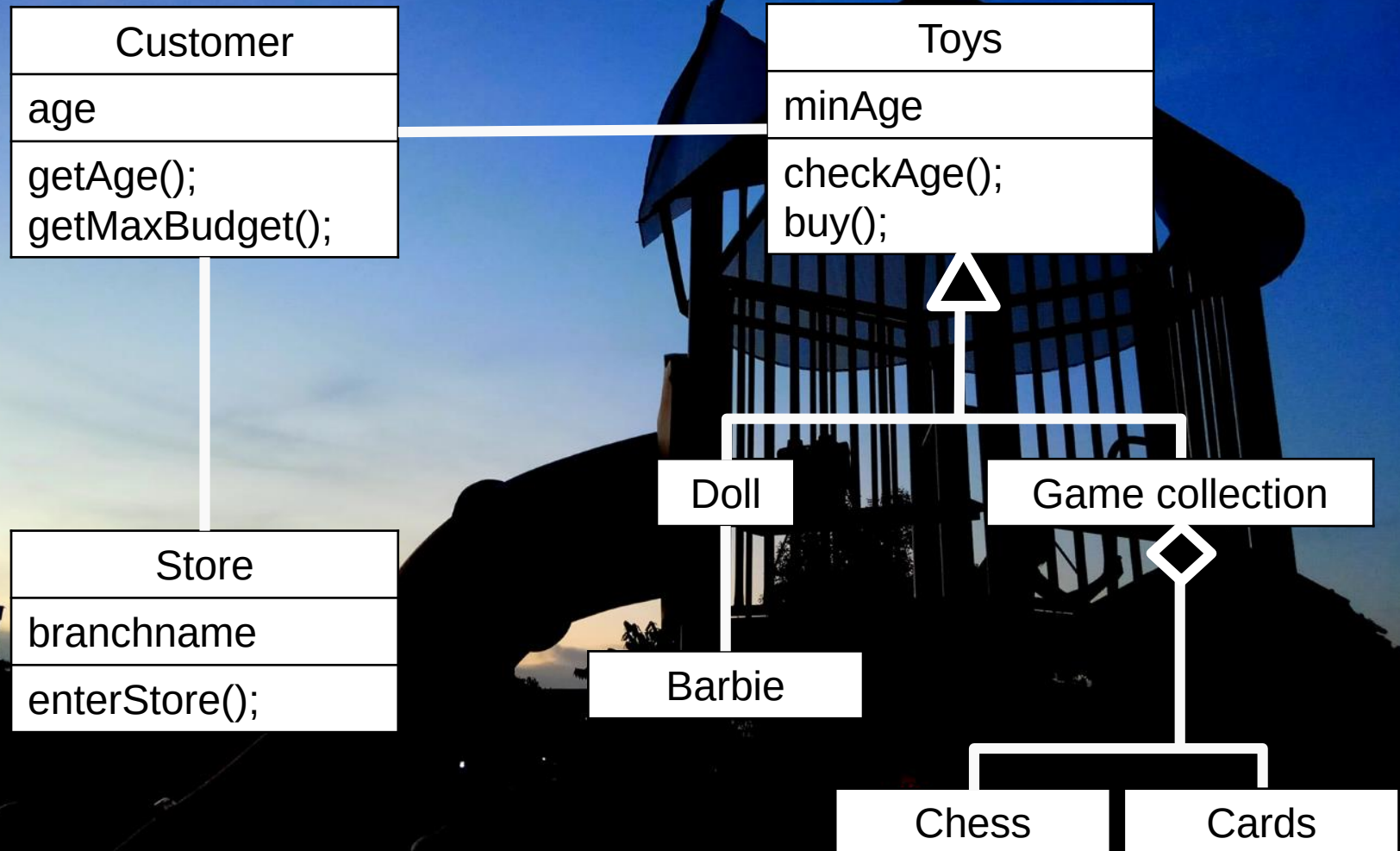
Overview of analysis

Identified item	Grammatical construct	UML Component
“Doll” “Game collection”, “customer”	Proper noun Common noun	Object instance Class
“10 year old”, “50 Ringgit”	Adjective	Attribute
“enter” “recommends” “suggests”	Verb	Method
“type of”	Classifier	Inheritance
“Consists of”	Having verb	Aggregation
“less than” “must be”	Modal verb	Constraints



Analysis Object model

- Draft Class Diagram

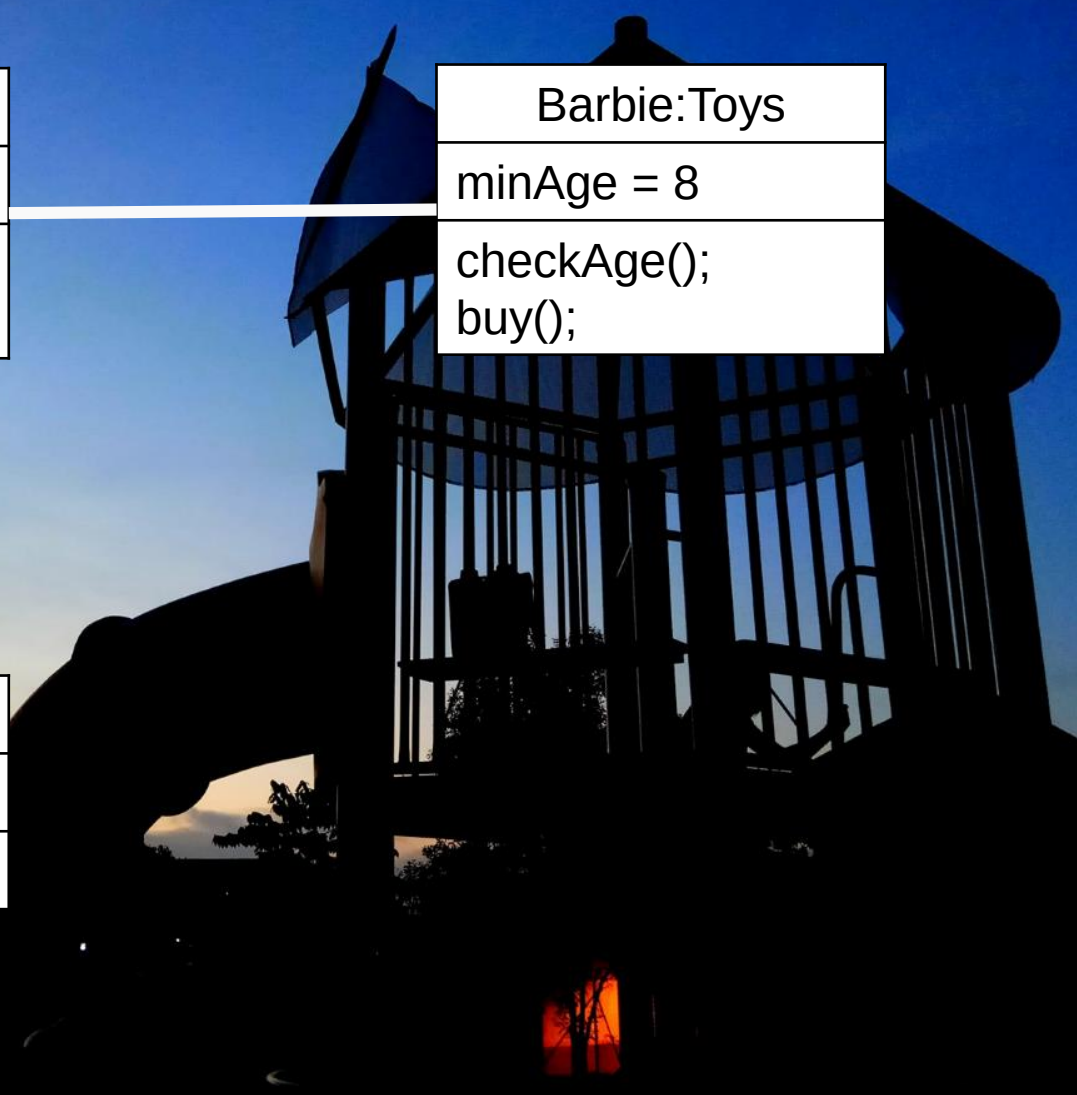


- Draft Object (instance) Diagram

Mike: Customer
age = 10
getAge(); getMaxBudget();

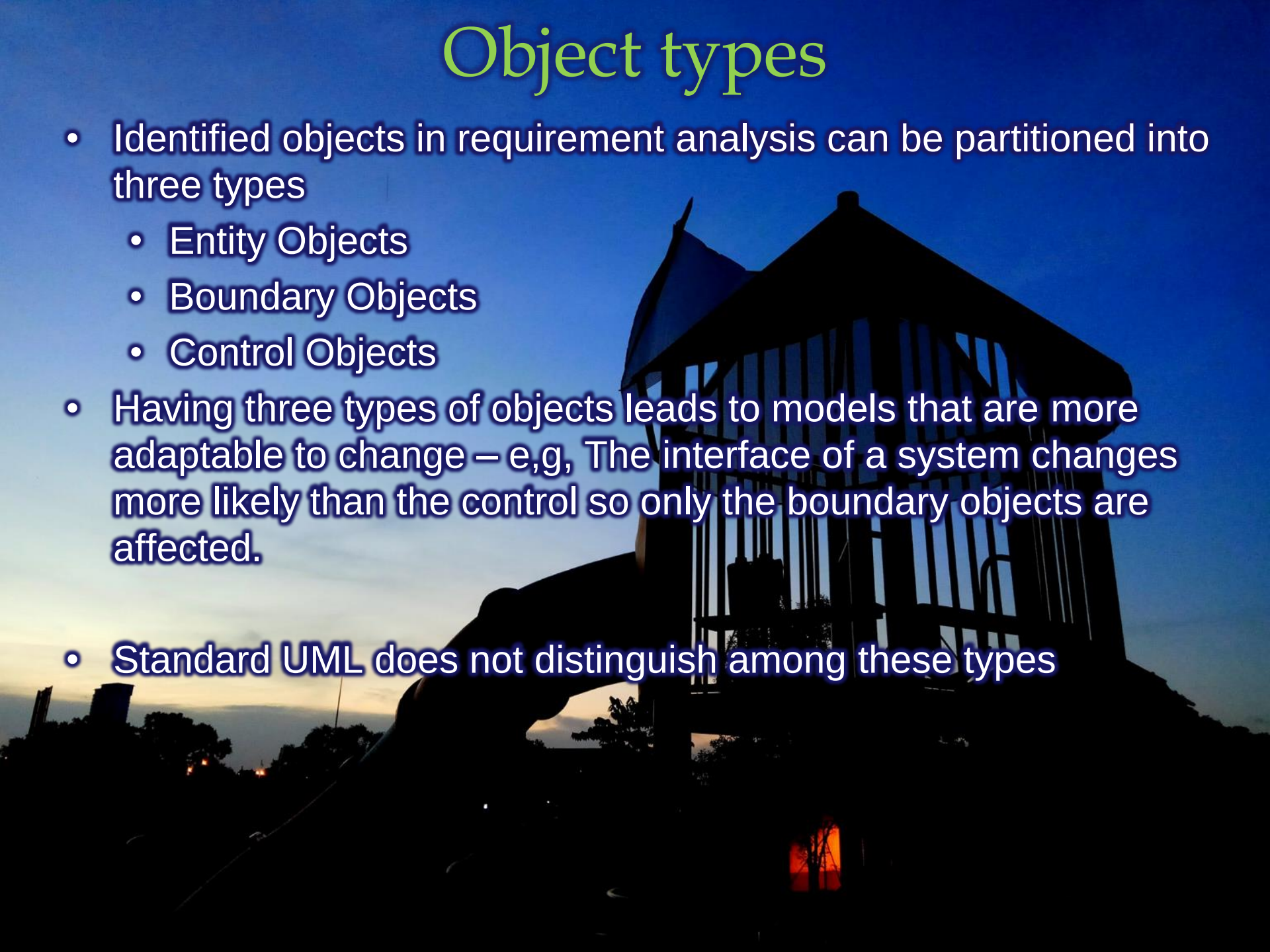
Barbie:Toys
minAge = 8
checkAge(); buy();

Toy'sRUs:Store
branchname
enterStore();



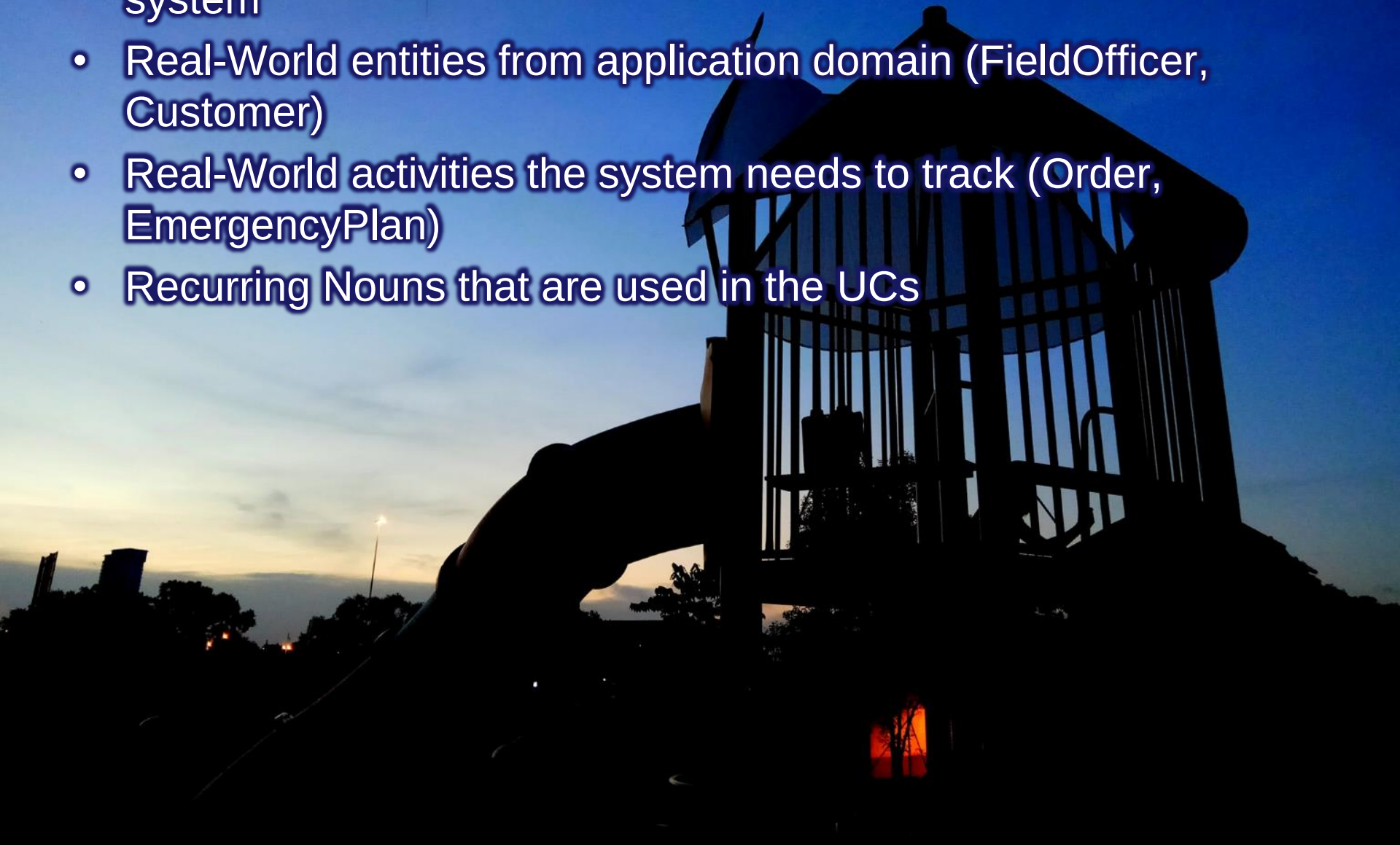
Object types

- Identified objects in requirement analysis can be partitioned into three types
 - Entity Objects
 - Boundary Objects
 - Control Objects
- Having three types of objects leads to models that are more adaptable to change – e.g, The interface of a system changes more likely than the control so only the boundary objects are affected.
- Standard UML does not distinguish among these types



Entity objects

- Represents persistent or long-lived information tracked by the system
- Real-World entities from application domain (FieldOfficer, Customer)
- Real-World activities the system needs to track (Order, EmergencyPlan)
- Recurring Nouns that are used in the UCs

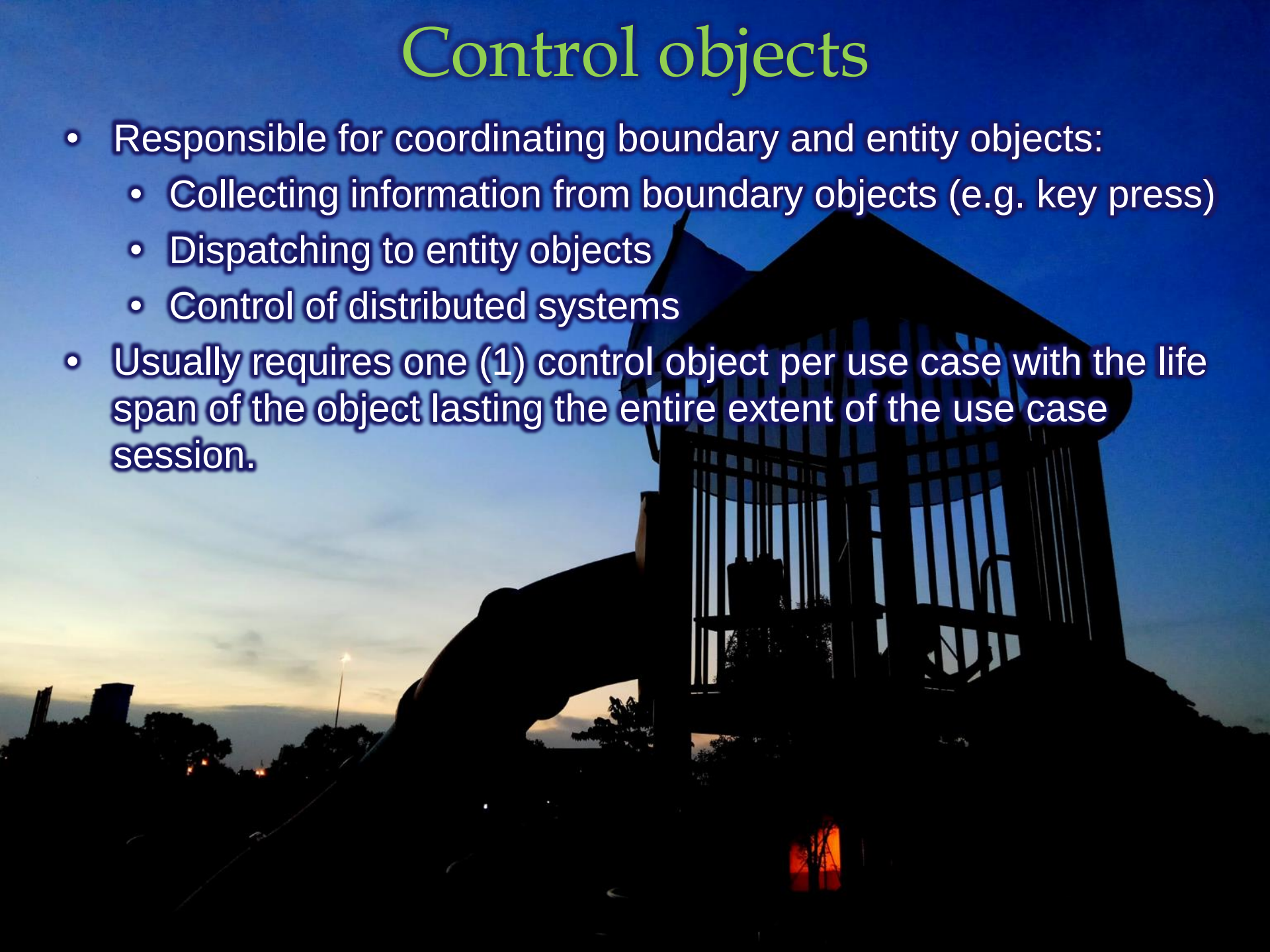


Boundary objects

- Represent the interaction between the user and the system
- Closely related to objects at the system boundary
- Represent the system interface for the actors
- Forms, Messages, Terminals for different users
- Identify Forms the user needs to enter data into the system
- Identify messages the system uses to respond to the user
- Also represent interface for external systems (e.g. online payment)
- Described in End User Terms
 - Good: EmergencyReportForm, ReportButton
 - Bad: MenuItem, ScrollBar (too detailed)

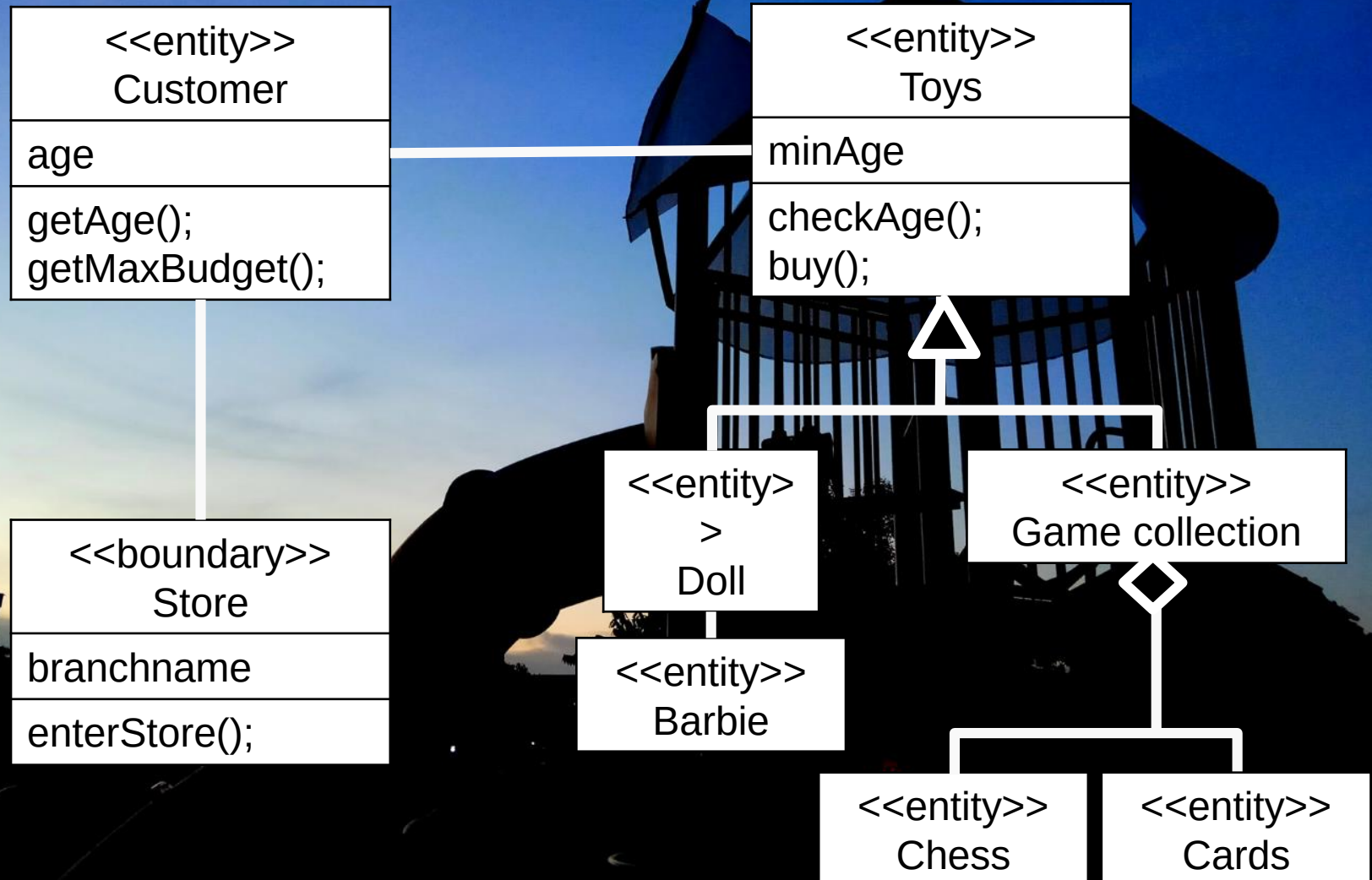
Control objects

- Responsible for coordinating boundary and entity objects:
 - Collecting information from boundary objects (e.g. key press)
 - Dispatching to entity objects
 - Control of distributed systems
- Usually requires one (1) control object per use case with the life span of the object lasting the entire extent of the use case session.



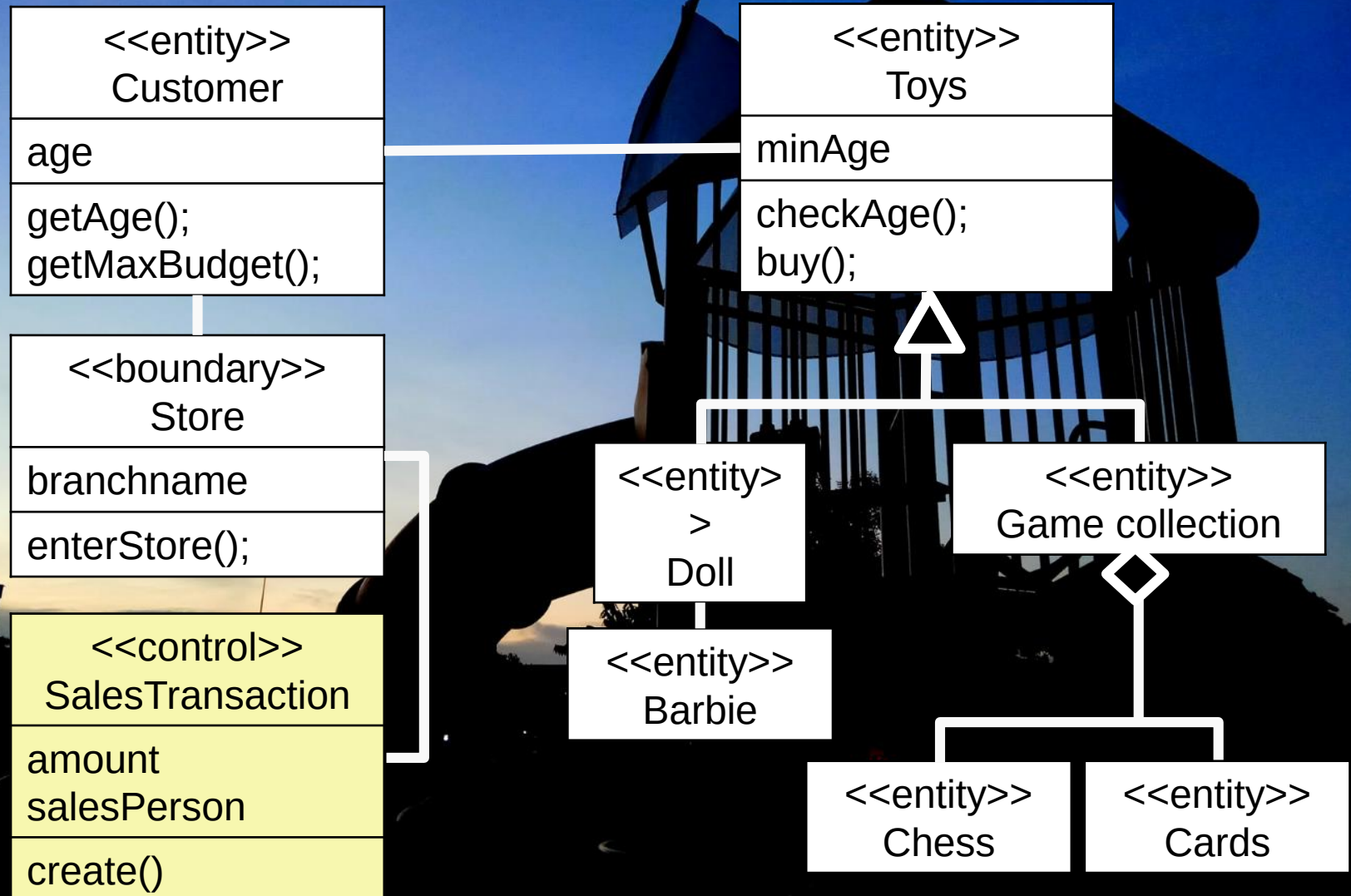
Analysis Object model

- Extension to existing class diagram



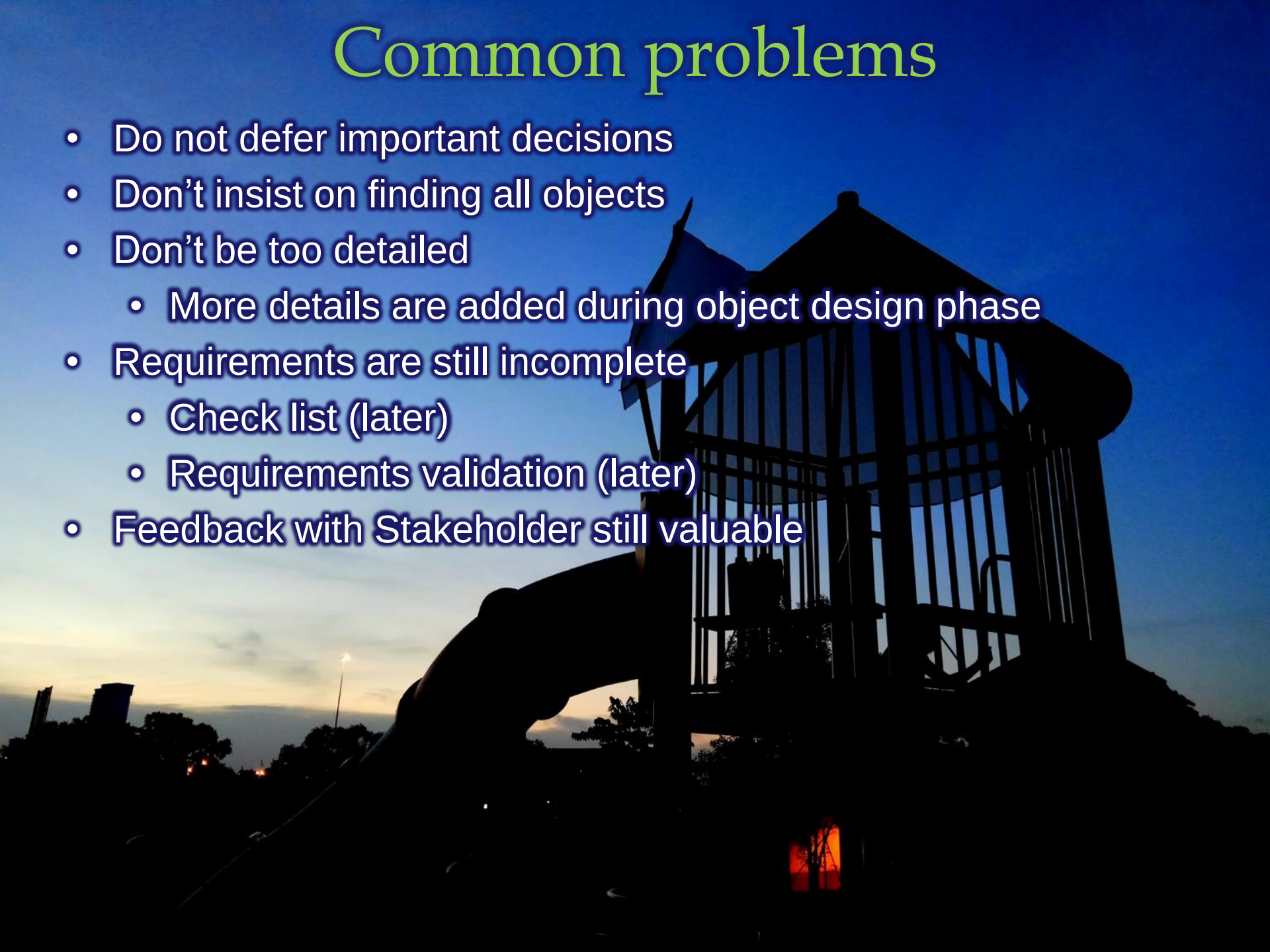
Analysis Object model

- Extension to existing class diagram



Common problems

- Do not defer important decisions
- Don't insist on finding all objects
- Don't be too detailed
 - More details are added during object design phase
- Requirements are still incomplete
 - Check list (later)
 - Requirements validation (later)
- Feedback with Stakeholder still valuable



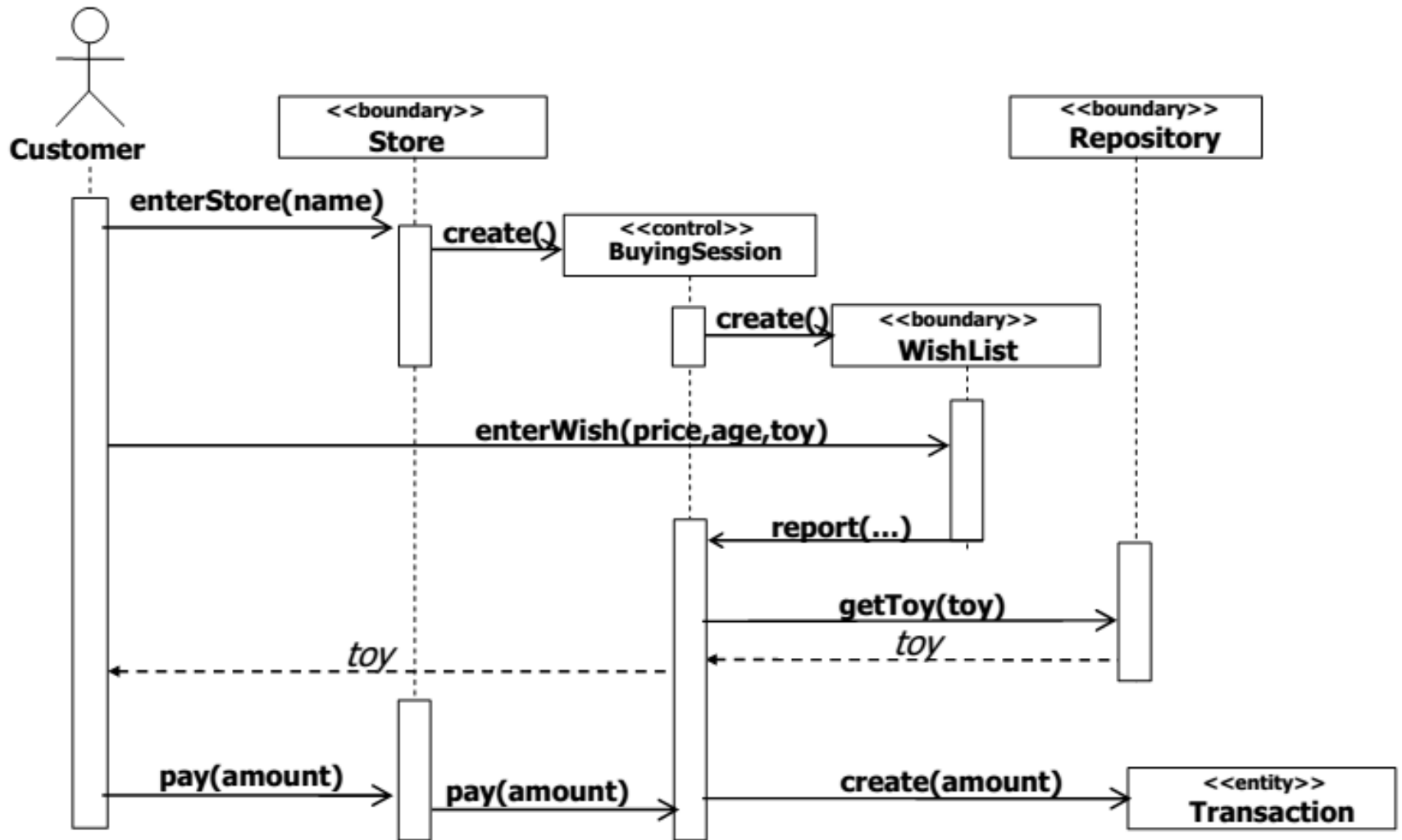
Dynamic models

- Modeling of behavioral aspects among the participating objects
- Tie use cases with objects
 - Shows how the behavior of a use case (or scenario) is distributed among its participating objects
 - Identify new objects and classes
 - Helps to determine how decentralized the system is.
- Purpose:
 - Detect and supply methods for the object model
- How do we do this?
 - Start with use case or scenario
 - Model interaction between objects - sequence diagram, CRC
 - Model dynamic behavior of a single object - state chart diagram

Dynamic modeling with UML

- Includes:
 - Sequence Diagram
 - Can be read out of use cases
 - Dynamic behavior of a set of objects arranged against a time sequence
 - Good for real-time specifications and complex scenarios
 - Information of the life time of objects
 - Also show relationships between objects methods
 - State Chart Diagram
 - describes the dynamic behavior of a single object
 - A state machine that describes the response of an object of a given class to the receipt of outside stimuli (Events)

Sequence Diagram Example



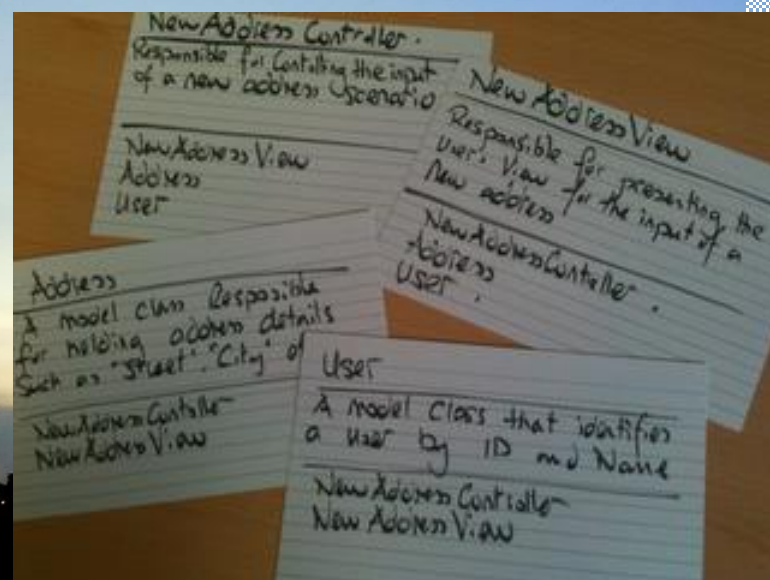
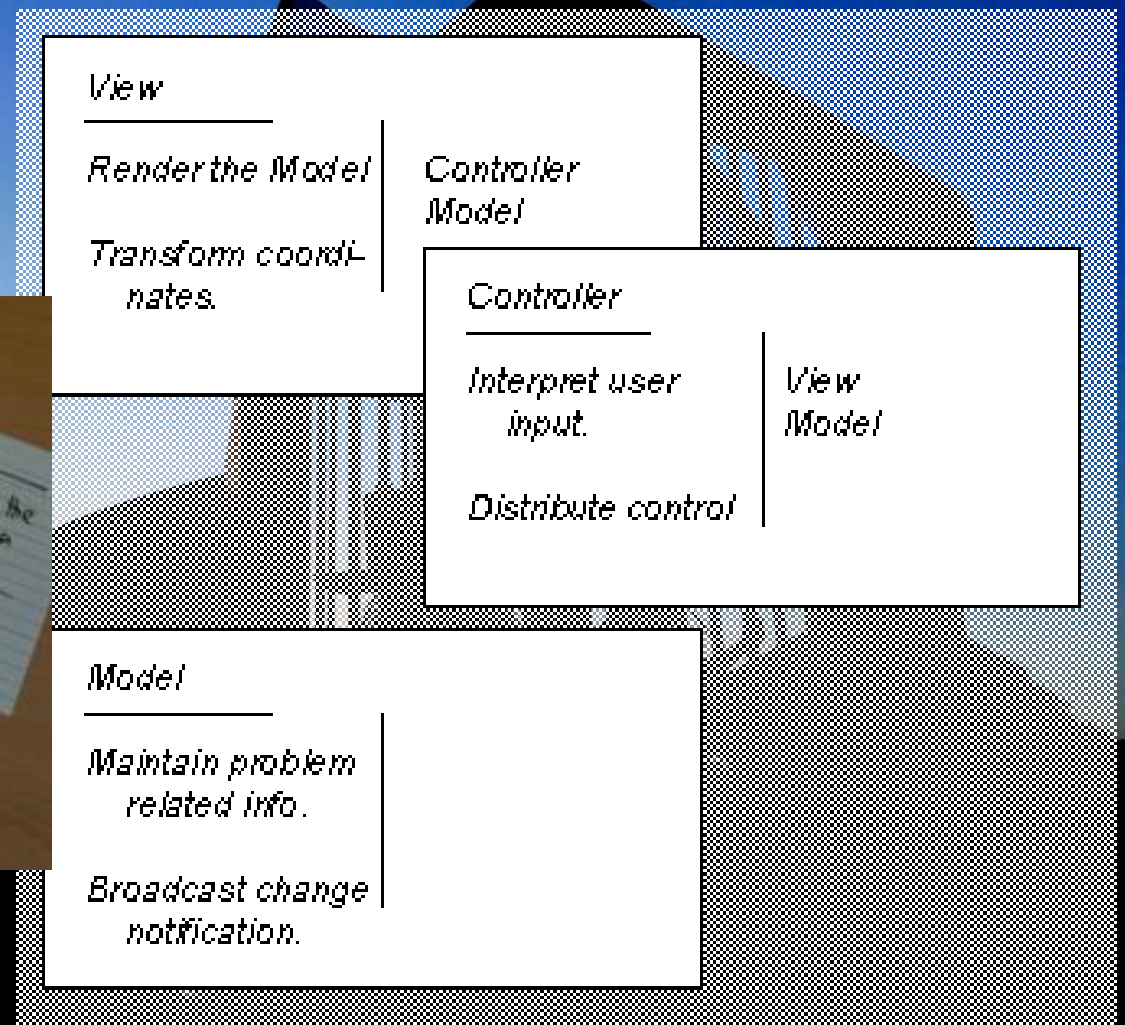
- Sequence diagrams will be covered in detail later

CRC

- CRC Cards are valuable tools in OO to represent classes, responsibilities and collaborators using cards
- One card represents one class with the name written at the top of the card
- Each class has a limited set of well defined responsibilities written on the left column
- The CRC Card also lists the collaborators of each class on the right column
- The attributes of a class can be recorded at the back of the CRC card.
- Using a card helps to limit the number of methods (responsibilities) and collaborators a class can have to keep a class' design at a minimum

Class Name	
Responsibilities	Collaborators

Student	
Student number	Seminar
Name	
Address	
Phone number	
Enroll in a seminar	
Drop a seminar	
Request transcripts	



Class Name:

Superclasses:

Subclasses:

Responsibilities

Collaborators

Vehicle

Description

A trans

Attributes

Registration

Car

Superclasses: Vehicle

Subclasses

Responsibilities

Start

Stop

Passenger

Superclasses: Person

Subclasses:

Responsibilities:

Drive

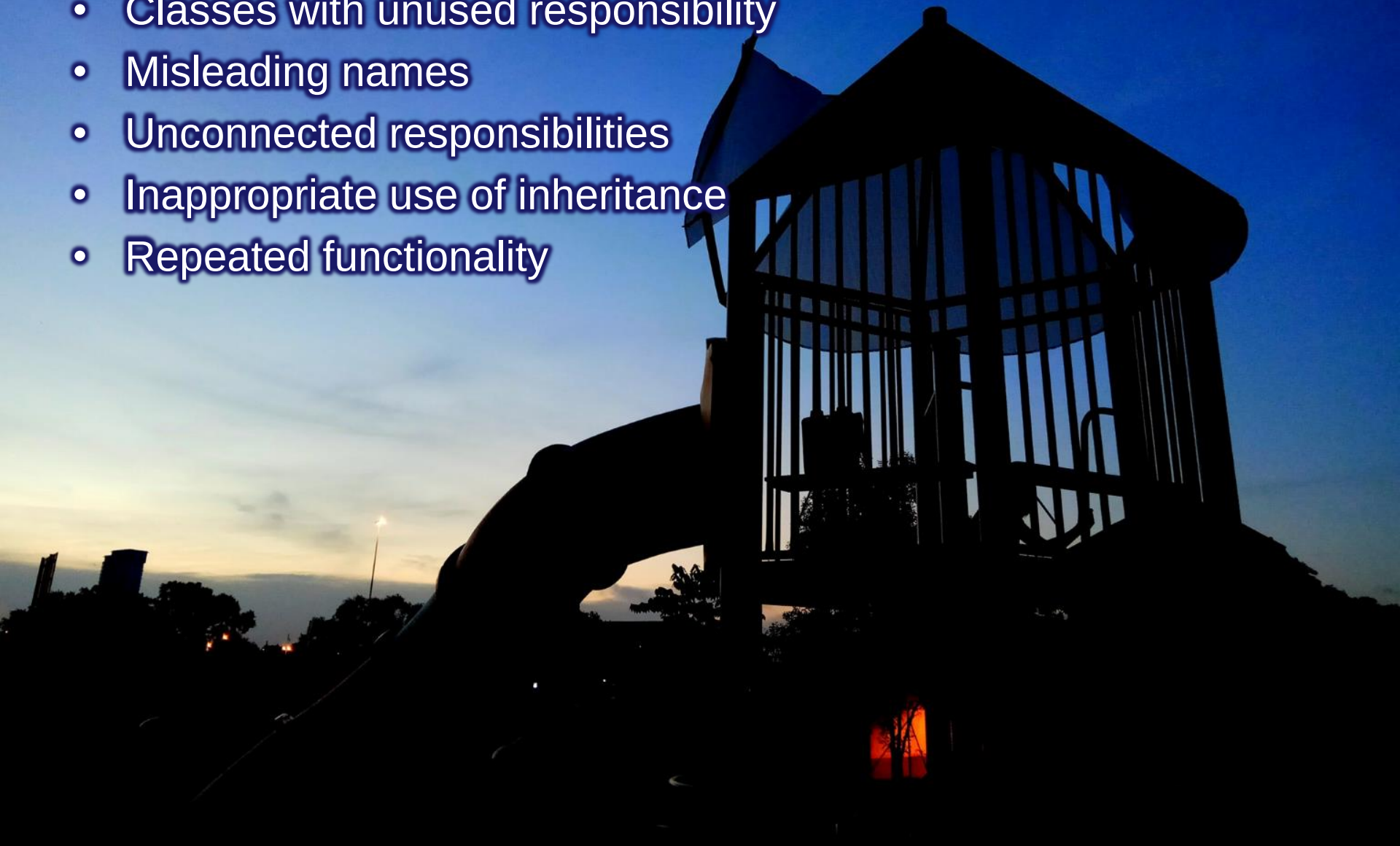
BuckleUp

Collaborators:

Car

Common class design flaws

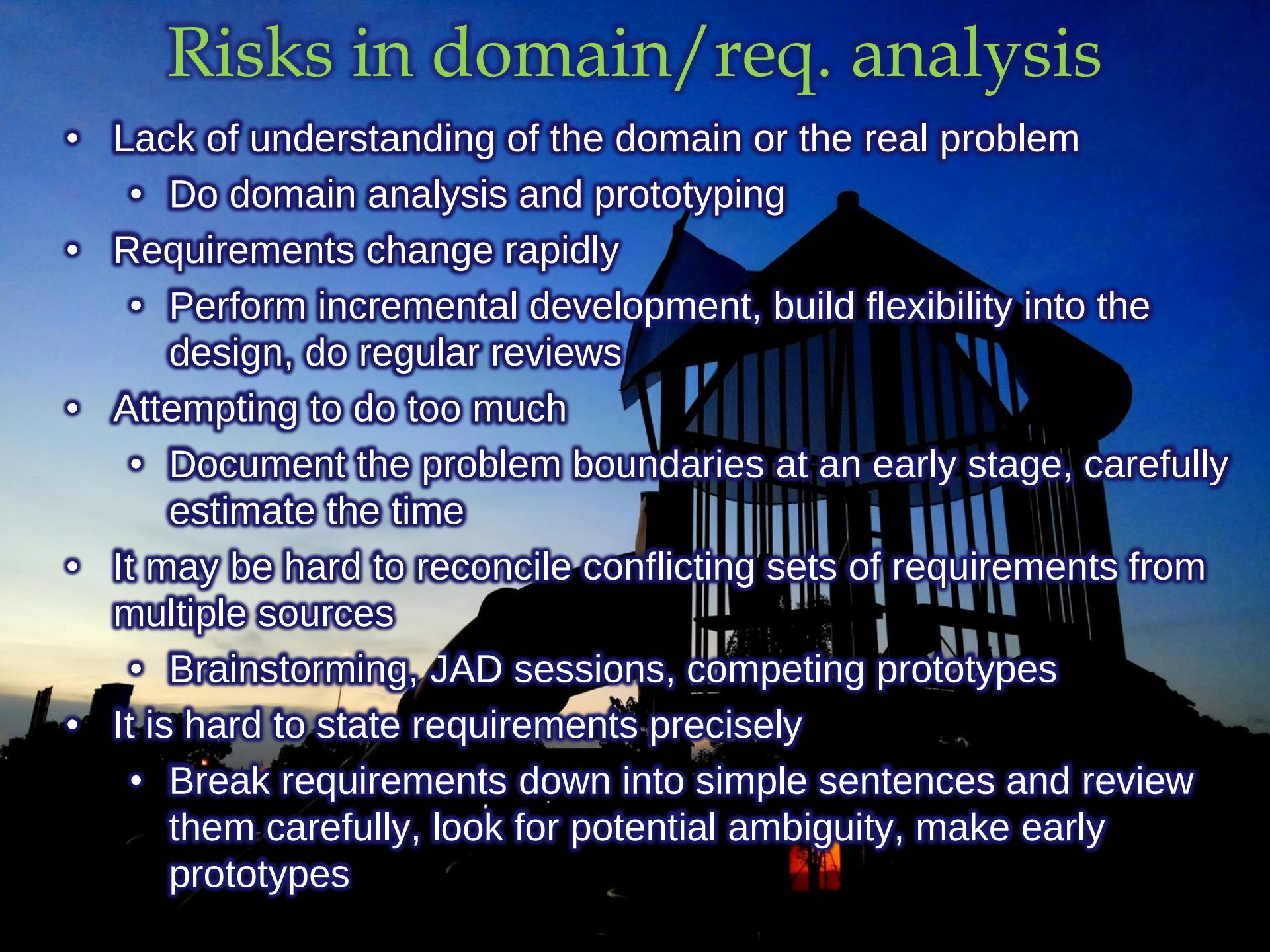
- Classes with too much responsibilities/no responsibility
- Classes with unused responsibility
- Misleading names
- Unconnected responsibilities
- Inappropriate use of inheritance
- Repeated functionality



Tips in dynamic modeling

- It is not necessary (and sometimes impossible) to construct the dynamic model for the entire problem domain.
- Construct dynamic models only for classes with significant dynamic behavior – e.g. those that require specific flow of operations or sequence
- Consider only relevant attributes - Use abstraction wherever Possible
- Look at the granularity of the application when deciding on actions and activities (e.g. is “Perform sales transaction” better than “Take item”, “Scan item barcode”, “Retrieve relevant item price”, “Add to total”, “repeat”)

Risks in domain/req. analysis

- Lack of understanding of the domain or the real problem
 - Do domain analysis and prototyping
 - Requirements change rapidly
 - Perform incremental development, build flexibility into the design, do regular reviews
 - Attempting to do too much
 - Document the problem boundaries at an early stage, carefully estimate the time
 - It may be hard to reconcile conflicting sets of requirements from multiple sources
 - Brainstorming, JAD sessions, competing prototypes
 - It is hard to state requirements precisely
 - Break requirements down into simple sentences and review them carefully, look for potential ambiguity, make early prototypes
- 
- A silhouette of a traditional East Asian pagoda with multiple tiers and a pointed roof, set against a bright sunset sky. The pagoda is dark, with its structural details visible. The sky transitions from a deep blue at the top to a bright orange and yellow near the horizon. The overall mood is serene and contemplative.

Summary

- What is the structure of the system?
 - Create class diagrams
 - Identify objects.
 - What are the associations between them? What is their multiplicity? What are the attributes of the objects? What operations are defined on the objects?
- What is its behavior?
 - Create sequence diagrams
 - Identify senders and receivers
 - Show sequence of events exchanged between objects.
 - Identify event dependencies and event concurrency.
 - Create state diagrams
 - Only for the dynamically interesting objects.